

Setting Up your Goal

Single Number Evaluation Metric

Whether you're tuning hyperparameters, or trying out different ideas for learning algorithms, or just trying out different options for building your machine learning system. You'll find that your progress will be much faster if you have a single real number evaluation metric that lets you quickly tell if the new thing you just tried is working better or worse than your last idea.

Example: Recognize a cat

- **Precision:** When our model recognize a cat, What percentage actually are cat? When model predict this is a cat: There's a 95% chance it really is a cat
- **Recall:** What percentage of actual cats, are correctly recognized

There's often a tradeoff between precision and recall and you care about both.

But of all the images that are cats, you also want it to pull a large fraction of them as cats. So it might be reasonable to try to evaluate the classifiers in terms of its precision and its recall.

The problem with using precision recall as your evaluation metric is that if classifier A does better on recall, which it does here, the classifier B does better on precision, then you're not sure which classifier is better.

And if you're trying out a lot of different ideas, a lot of different hyperparameters, you want to rather quickly try out not just two classifiers, but maybe **a dozen classifiers** and quickly pick out the, quote, "best ones", so you can keep on iterating from there.

And with two evaluation metrics, it is difficult to know how to quickly pick one of the two or quickly pick one of the ten. So what I recommend is rather than using two numbers, precision and recall, to pick a classifier, you just have to find a new evaluation metric that combines precision and recall.

F_1 Score = "Average" of P and R

→ Harmonic mean: $\frac{2}{\frac{1}{P} + \frac{1}{R}}$

Using a single number evaluation metric

Idea → Experiment → Code → Idea

Classifier	Precision	Recall	F1 Score
A	95%	90%	92.4%
B	98%	85%	91.0%

Handwritten notes:

- Of examples recognized as cat, what % actually are cats? (Precision)
- What % of actual cats are correctly recognized? (Recall)
- F_1 score = "Average" of P and R.
- $\left(\frac{2}{\frac{1}{P} + \frac{1}{R}} \right)$ "Harmonic mean"
- Dev set + Single number evaluation metric
- ↑ real speed up iterating

Andrew Ng

Let's look at another example. Let's say you're building a cat app for cat lovers in four major geographies, the US, China, India, and other, the rest of the world

And let's say that your two classifiers achieve different errors in data from these four different geographies. So algorithm A achieves **3% error** on pictures submitted by US users and so on. So it might be reasonable to keep track of how well your classifiers do in these different markets or these different geographies.

Another example

Algorithm	US	China	India	Other	Average
A	3%	7%	5%	9%	6%
B	5%	6%	5%	10%	6.5%
C	2%	3%	4%	5%	3.5%
D	5%	8%	7%	2%	5.25%
E	4%	5%	2%	4%	3.75%
F	7%	11%	8%	12%	9.5%

If you have to pick an algorithm
to keep on iterating from.

Andrew Ng

Look at the table of 4 columns (US, China, India, Other) It's hard for us to choose which is the best algorithm → try using average and pick the algorithm reached the lowest one.

Satisficing and Optimizing Metric

Another cat classification example

Classifier	Accuracy	Running time
A	90%	80ms
B	92%	95ms
C	95%	1,500ms

$$\text{Cost} = \text{accuracy} - 0.5 \times \text{Running Time}$$

maximize accuracy
subject to Running Time $\leq$ 100 ms.

N metrics: 1 optimizing
N-1 satisfying

Here's another example.

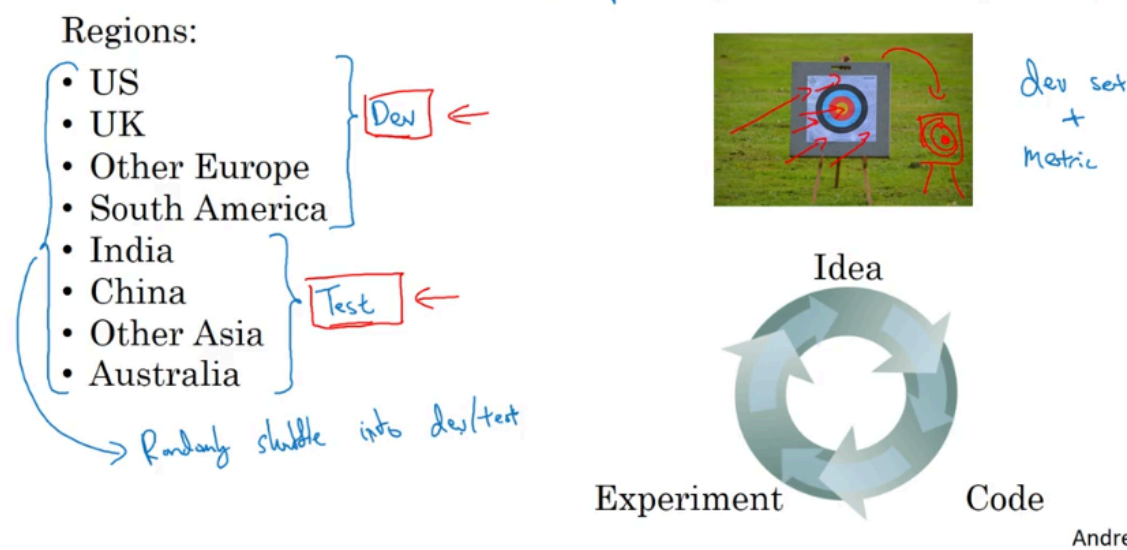
Andrew Ng

Given that we have 2 metrics:

- Accuracy and Running time: We should choose one for **optimizing** and rest for **satisfying**
- For the optimizing: → choose the best one
- For the satisfying: → choose threshold

Train/Dev/Test Distributions

Cat classification dev/test sets



So that, both the dev and test sets have data from all eight regions

Assume that:

- We have a dev set from regions: US, UK, ...
- And a Test set from regions: India, China, ...

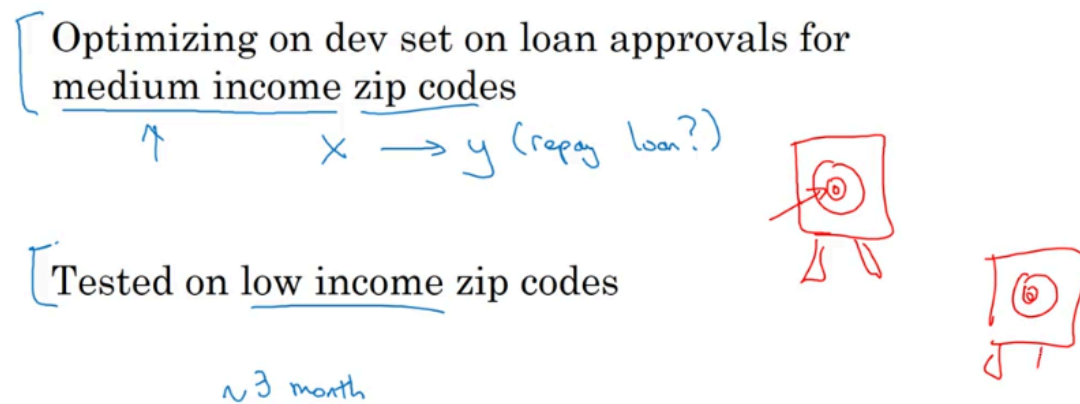
Because, what happen once you've established that dev set and the metric is that, the team can innovate very quickly, try different ideas, run experiments and very quickly use the dev set and the metric to evaluate crossfires and try to pick the best one.

So, machine learning teams are often very good at shooting different arrows into targets and innovating to get closer and closer to hitting the bullseye. So, doing well on your metric on your dev sets. And, the problem with how we've set up the dev and test sets in the example on the left is that, your team might spend months innovating to do well on the dev set only to realize that, when you finally go to test them on the test set, that data from these four countries or these four regions at the bottom, might be very different than the regions in your dev set.

So, you might have a nasty surprise and realize that, all the months of work you spent optimizing to the dev set, is not giving you good performance on the test set.

So, having dev and test sets from different distributions is like setting a target, having your team spend months trying to aim closer and closer to bull's eye, only to realize after months of work that, you'll say, "Oh wait, to test it, I'm going to move target over here." And, the team might say, "Well, why did you make us spend months optimizing for a different bull's eye when suddenly, you can move the bull's eye to a different location somewhere else?" So, to avoid this, what I recommend instead is that, [you take all this randomly shuffled data into the dev and test set](#). So that, both the dev and test sets have data from all eight regions and that the dev and test sets really come from the same distribution, which is the distribution of all of your data mixed together.

True story (details changed)



Andrew Ng

This is a totally different location.

I know a machine learning team that actually spent several months optimizing on a dev set which was comprised of loan approvals for medium income zip codes.

So, the specific machine learning problem was, "Given an input X about a loan application, can you predict y and which is, whether or not, they'll repay the loan?" So, this helps you decide whether or not to approve a loan. And so, the dev set came from loan applications. They came from medium income zip codes. Zip codes is what we call postal codes in the United States. But, after working on this for a few months, the team then, suddenly decided to test this on data from low income zip codes or low income postal codes.

And, of course, the distributional data for medium income and low income zip codes is very different. And, the crossfire, that they spend so much time optimizing in the former case, just didn't work well at all on the latter case.

And so, this particular team actually wasted about three months of time and had to go back and really re-do a lot of work. And, what happened here was, the team spent three months aiming for one target, and then, after three months, the manager asked, "Oh, how are you doing on hitting this other target?" This is a totally different location. And, it just was a very frustrating experience for the team. So, what I recommend for setting up a dev set and test set is, **choose a dev set and test set to reflect data you expect to get in future and consider important to do well on.**

Guideline

- Choose a dev set and test set to reflect data you expect to get in the future and consider important to do well on.
- known that: The dev set and test set should in a same distribution

Size of the Dev and Test Sets

Size of test set:

- Set your test set to be big enough to give high confidence in the overall performance of your system

And the rule of thumb is really to try to set the dev set to big enough for its purpose, which helps you evaluate different ideas and pick this up A or B is better. And the purpose of test set is to help you evaluate your final cost buys. You just have to set your test set big enough for that purpose, and that could be much less than 30% of the data.

When to Change Dev/Test Sets and Metrics?

Cat dataset examples

Metric: Classification error

- Algorithm A: 3% error → but let pornographic images through
- Algorithm B: 5% error → dont let pornographic (khiêu dâm) images through

User and company will like Algorithm B more

Cat dataset examples

Metric + Dev : Prefer A
You/users : Prefer B.

Metric: classification error

Algorithm A: 3% error → pornographic

✓ Algorithm B: 5% error

Error:
$$\frac{1}{m_{dev}} \sum_{i=1}^{m_{dev}} w^{(i)} \mathbb{I}\{y_{pred}^{(i)} \neq y^{(i)}\}$$

$w^{(i)} = \begin{cases} 1 & \text{if } x^{(i)} \text{ is non-porn} \\ 10 & \text{if } x^{(i)} \text{ is porn} \end{cases}$

$\mathbb{I}\{y_{pred}^{(i)} \neq y^{(i)}\}$ ← predicted value (0/1)

use ten time bigger weight to classify pornographic images correctly

The point is that if you're not satisfied with your old error metric then don't keep coasting with an error metric you're unsatisfied with, instead try to define a new one that you think better captures your preferences in terms of what's actually a better algorithm

Orthogonalization for cat pictures: anti-porn

- 1. So far we've only discussed how to define a metric to evaluate classifiers. ← Place target ⌋
- 2. Worry separately about how to do well on this metric. ⌋
← Aim (shoot at target)

→
$$J = \frac{1}{\sum w^{(i)}} \sum_{i=1}^m w^{(i)} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$



Andrew Ng

Another example

Algorithm A: 3% error

✓ Algorithm B: 5% error ←

→ Dev/test



→ User images



If doing well on your metric + dev/test set does not correspond to doing well on your application, change your metric and/or dev/test set.

Andrew Ng

So my recommendation is, even if you can't define the perfect evaluation metric and dev set, just set something up quickly and use that to drive the speed of your team iterating.